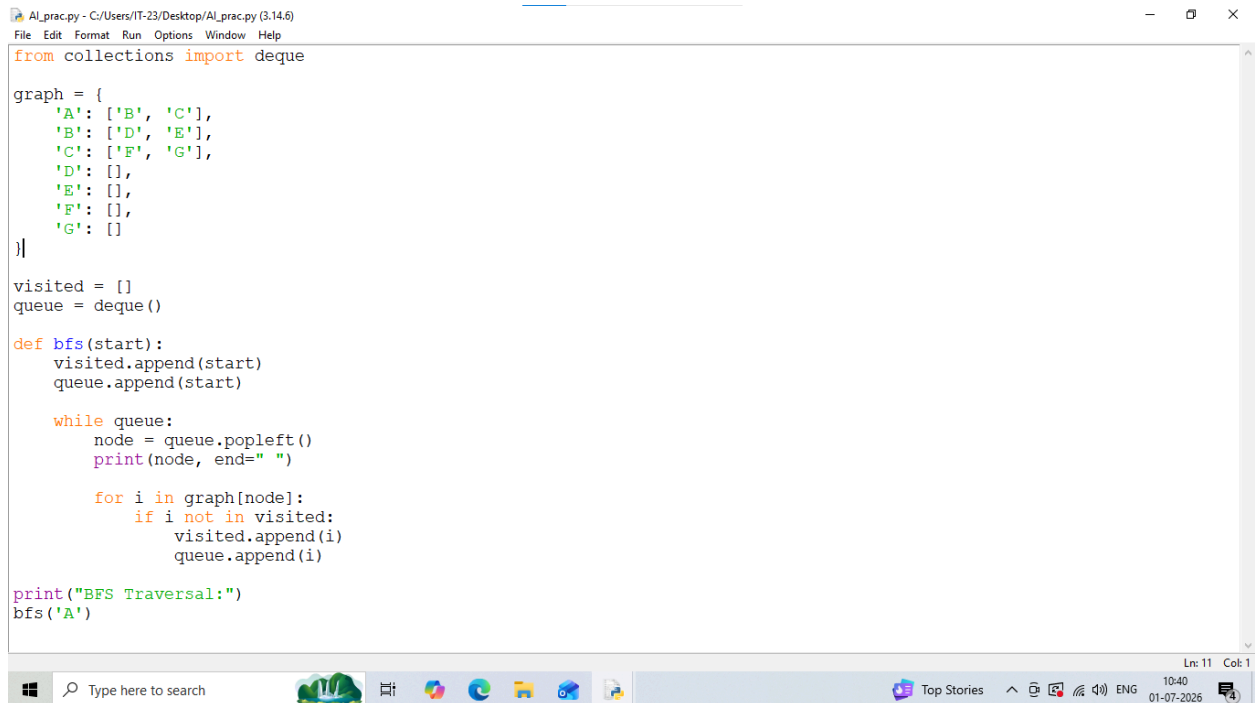


AIM : Breadth First Search & Iterative Depth First Search in Python.

- A) Implement the Breadth First Search algorithm to solve a given problem.
- B) Implement the Iterative Depth First Search algorithm to solve the same problem.
- C) Compare the performance and efficiency of both algorithms.

Breadth First Search



```
AI_prac.py - C:/Users/IT-23/Desktop/AI_prac.py (3.14.6)
File Edit Format Run Options Window Help
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

visited = []
queue = deque()

def bfs(start):
    visited.append(start)
    queue.append(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")

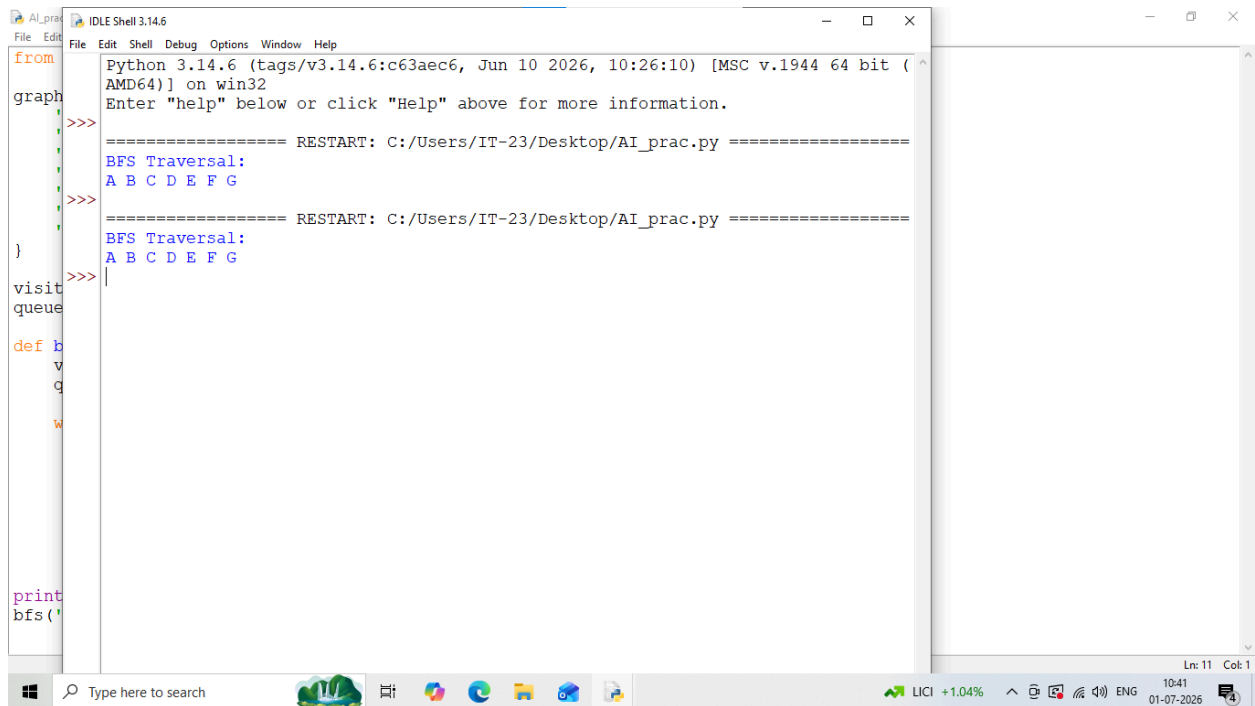
        for i in graph[node]:
            if i not in visited:
                visited.append(i)
                queue.append(i)

print("BFS Traversal:")
bfs('A')
```

The screenshot shows a Windows 10 desktop with a code editor window titled "AI_prac.py - C:/Users/IT-23/Desktop/AI_prac.py (3.14.6)". The editor contains a Python script for Breadth First Search (BFS). The script defines a graph with nodes A through G and their connections. It uses a deque as a queue and a list for visited nodes. The bfs function is defined to take a start node, add it to the queue and visited list, and then iteratively process nodes until the queue is empty. The script prints the BFS Traversal starting from node A. The Windows taskbar at the bottom shows the Start button, search bar, task view, and several application icons. The system tray on the right shows the date and time as 10:40 on 01-07-2026.

SHETH L.U.J. AND SIR M.V. COLLEGE OF ARTS, SCIENCE AND COMMERCE
SUBJECT : ARTIFICIAL INTELLIGENCE

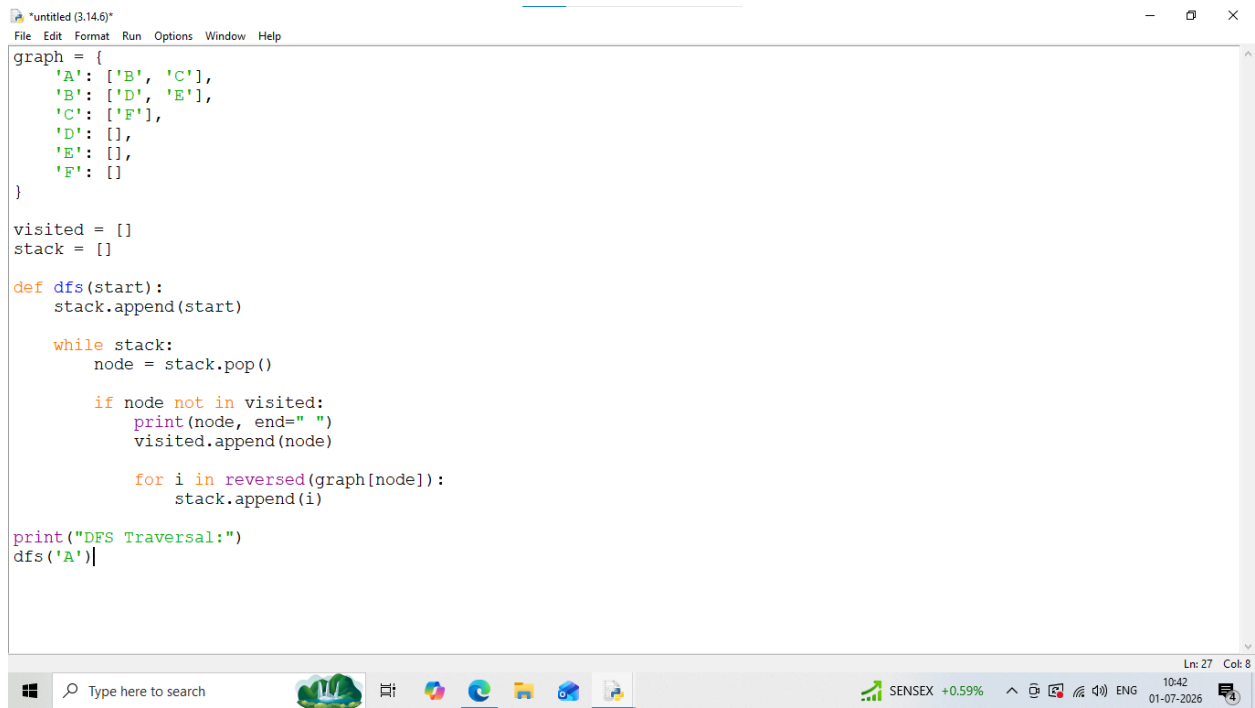
Output :



```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/IT-23/Desktop/AI_prac.py =====
BFS Traversal:
A B C D E F G
>>> ===== RESTART: C:/Users/IT-23/Desktop/AI_prac.py =====
BFS Traversal:
A B C D E F G
>>>
```

Iterative Depth First Search



```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

visited = []
stack = []

def dfs(start):
    stack.append(start)

    while stack:
        node = stack.pop()

        if node not in visited:
            print(node, end=" ")
            visited.append(node)

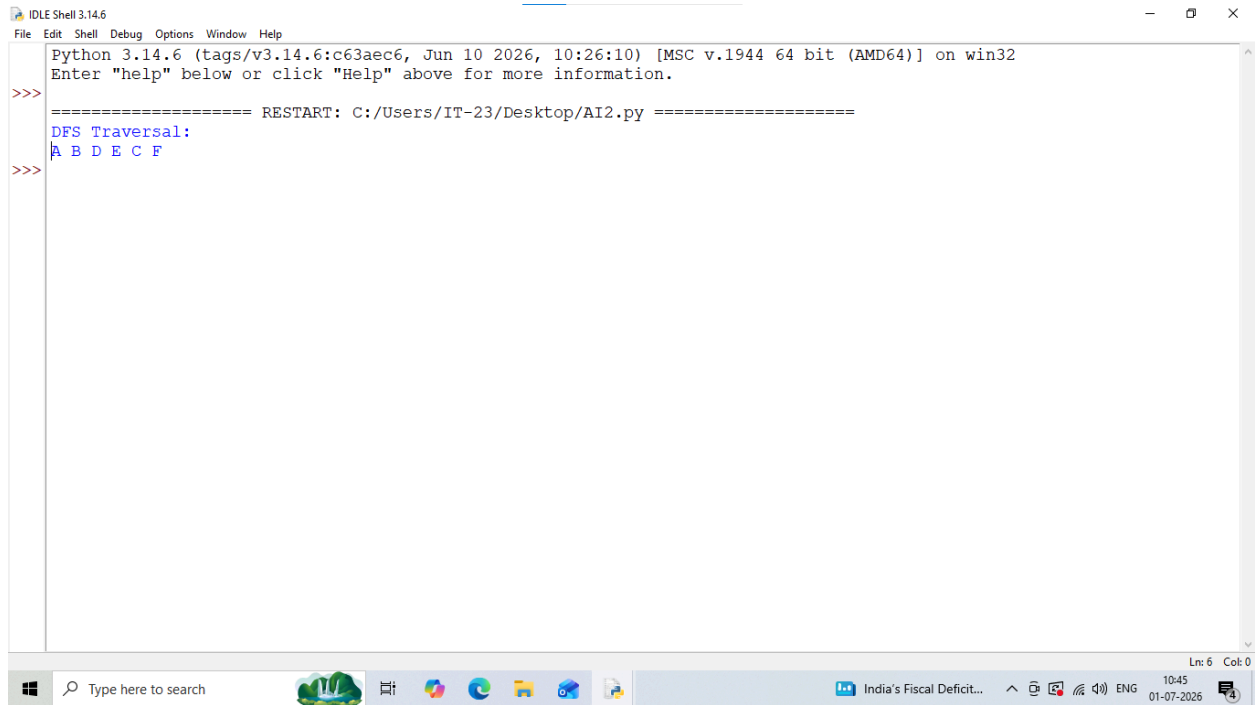
            for i in reversed(graph[node]):
                stack.append(i)

print("DFS Traversal:")
dfs('A')
```

Name : Sumeet Yadav
Roll No : T124

SHETH L.U.J. AND SIR M.V. COLLEGE OF ARTS, SCIENCE AND COMMERCE
SUBJECT : ARTIFICIAL INTELLIGENCE

OUTPUT :



```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/IT-23/Desktop/AI2.py =====
DFS Traversal:
A B D E C F
>>>
```

The screenshot shows a Python IDLE Shell window. The title bar reads 'IDLE Shell 3.14.6'. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area displays the Python interpreter's startup message, followed by a prompt '>>>'. The user has entered a command that restarts the shell and executes a DFS Traversal program. The output of the program is 'A B D E C F'. The status bar at the bottom shows 'Ln: 6 Col: 0'.

Name : Sumeet Yadav
Roll No : T124

C) Performance and Efficiency Comparison

Feature	Breadth First Search (BFS)	Iterative Depth First Search (DFS)
Data Structure	Queue (FIFO)	Stack (LIFO)
Traversal Order	Level by level	Goes as deep as possible before backtracking
Time Complexity	$O(V + E)$	$O(V + E)$
Space Complexity	$O(V)$	$O(V)$
Finds Shortest Path	Yes (in unweighted graphs)	No
Memory Usage	Higher for wide graphs	Usually lower for wide graphs
Suitable For	Shortest path, level-order traversal	Path finding, maze solving, topological traversal